

ReaCustomMenu

User Manual

Compatibility: Windows only

ReaCustomMenu is currently available for **Windows only**. There is currently no macOS or Linux version.

1. What is ReaCustomMenu for?

ReaCustomMenu is an extension for **REAPER** that allows you to create your own menus directly in REAPER's main menu bar.

The menus are defined in a simple XML configuration file named:

ReaCustomMenu.xml

ReaCustomMenu allows you to:

- create one or more top-level menus;
- create submenus, with no practical depth limitation;
- add native REAPER actions;
- add ReaScripts;
- add SWS actions;
- insert separators;
- assign custom display names to scripts and other named commands.

The purpose is to provide a simple way of creating a personal command and script organization without modifying REAPER's menus through the **Customize Menus/Toolbars** system.

Important

The XML file is read when REAPER loads the extension.

After modifying ReaCustomMenu.xml, REAPER must be restarted for the changes to take effect.

2. Installation

2.1. Components

ReaCustomMenu is deliberately very simple.

The distribution contains:

- **reaper_ReaCustomMenu.dll** — the extension;
- **ReaCustomMenu.xml** — the basic configuration file.

The supplied XML file is intentionally minimal:

```
<?xml version="1.0" encoding="utf-8"?>
<menus></menus>
```

This is a perfectly valid XML file, but it does not create any menus yet. **The user is expected to add their own menus and commands.**

The extension may also create:

- **ReaCustomMenu.log** — a log file mainly useful for troubleshooting.

2.2. Installing the DLL

Copy:

`reaper_ReaCustomMenu.dll`

to REAPER's:

`UserPlugins`

folder.

The easiest way to locate REAPER's resource folder is to use:

Options → Show REAPER resource path in explorer/finder

Then open:

`UserPlugins`

and copy the DLL there.

2.3. Installing the XML file

Place:

`ReaCustomMenu.xml`

directly in REAPER's **resource folder**, at the same level as the `UserPlugins` folder.

For example:

`C:\Users\<User>\AppData\Roaming\REAPER\ReaCustomMenu.xml`

The log file is also created in this folder:

`C:\Users\<User>\AppData\Roaming\REAPER\ReaCustomMenu.log`

The XML file is required to create menus, but **it is perfectly valid to leave it in its initial minimal state.**

3. Editing the XML file

3.1. What is XML?

If you have never used XML before, there is nothing complicated about it.

XML stands for:

eXtensible Markup Language

It is a structured text format used to describe information.

An XML document uses **tags** to describe its structure.

For example:

```
<menu name="My scripts">
    ...
</menu>
```

Here:

- `<menu>` marks the beginning of an element;
- `</menu>` marks its end;
- `name="My scripts"` is an **attribute**.

Some elements have no contents and can therefore be written in a shortened form:

```
<separator/>
```

In ReaCustomMenu, the XML elements correspond directly to menu elements.

3.2. Why use XML Notepad?

You can technically edit an XML file with any text editor, but this is not recommended for beginners.

XML Notepad, developed by Microsoft, provides a dedicated interface for viewing and editing XML documents. It displays the XML structure as a tree and can report XML errors while editing.

XML Notepad is freely available.

The official download is available from the project's **XML Notepad Releases** page on Microsoft's GitHub repository.

4. The structure of ReaCustomMenu.xml

The XML file supplied with ReaCustomMenu initially contains:

```
<?xml version="1.0" encoding="utf-8"?>
<menus></menus>
```

<menus> is the root element of the document.

To create a menu, simply add a <mainmenu> element inside it:

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<menus>
```

```
    <mainmenu name="Mixing" id="mixing">
```

```
        ...
```

```
    </mainmenu>
```

```
</menus>
```

A <mainmenu> has two attributes:

name

The name displayed in REAPER's main menu bar.

id

The internal identifier of the menu.

It must be **unique** within the XML file.

5. Adding an action

A native REAPER action can be added using its numeric command ID.

Example:

```
<action command="40044"/>
```

The number 40044 is the REAPER action ID.

If no **name** attribute is specified, REAPER automatically displays the action's normal name.

Therefore, for a native REAPER action, this is normally sufficient:

```
<action command="40044"/>
```

6. Adding a script

Scripts and other registered commands generally use identifiers such as:

```
_RSxxxxxxxxxxxxxxxxxxx
```

or:

`_SWSxxxxxxxxxxxxxxxxxxxx`

Example:

```
<action command="_RSxxxxxxxxxxxxxxxxxxxx"
      name="My script"/>
```

Important for scripts

For named commands such as `_RS . . .` or `_SWS . . .`, it is recommended to provide the `name` attribute.

Without `name`, REAPER may not have an action name to display and may display the command identifier instead.

It is therefore preferable to write:

```
<action command="_RSxxxxxxxxxxxxxxxxxxxx"
      name="Clean items"/>
```

The text specified in `name` is what will appear in the menu.

7. Adding a separator

A separator is simply:

```
<separator/>
```

For example:

```
<action command="40044"/>
```

```
<separator/>
```

```
<action command="40045"/>
```

This creates a separator line between the two actions.

8. Creating a submenu

A submenu uses the `<menu>` element.

Example:

```
<menu name="FX">
```

```
    <action command="40044"/>
```

```
    <separator/>
```

```
<action command="_RSxxxxxxxxxxxxxxxxxx"
      name="My script"/>
```

```
</menu>
```

This creates a submenu called **FX** containing the specified elements.

Submenus can themselves contain other submenus.

For example:

```
<menu name="Scripts">
```

```
  <menu name="Mixing">
```

```
    <action command="_RSaaaaaaaaaaaaaaaa"
          name="Script 1"/>
```

```
    <action command="_RSbbbbbbbbbbbbbbbbbb"
          name="Script 2"/>
```

```
  </menu>
```

```
</menu>
```

9. Complete example

Starting from the minimal XML file supplied with ReaCustomMenu, you could create:

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<menus>
```

```
  <mainmenu name="Mixing" id="mixing">
```

```
    <action command="40044"/>
```

```
    <separator/>
```

```
    <menu name="FX">
```

```
<action command="_RSxxxxxxxxxxxxxxxxx"
      name="My script"/>
```

```
<action command="_SWSxxxxxxxxxxxxxxxxx"
      name="An SWS command"/>
```

```
</menu>
```

```
</mainmenu>
```

```
<mainmenu name="MIDI" id="midi">
```

```
<action command="_RSaaaaaaaaaaaaaaaaa"
      name="MIDI Script 1"/>
```

```
<action command="_RSbbbbbbbbbbbbbbbbbb"
      name="MIDI Script 2"/>
```

```
</mainmenu>
```

```
<mainmenu name="Scripts" id="scripts">
```

```
<menu name="My scripts">
```

```
<action command="_RScccccccccccccccc"
      name="First script"/>
```

```
</menu>
```

```
</mainmenu>
```

```
</menus>
```

This creates three top-level menus:

- **Mixing**
- **MIDI**
- **Scripts**

with various submenus and actions.

10. Important rules

The XML must be valid

An opening tag must have a matching closing tag.

Correct:

```
<menu name="FX">  
    ...  
</menu>
```

Also correct:

```
<separator/>
```

Incorrect:

```
<menu name="FX">
```

with no corresponding `</menu>`.

XML Notepad makes these kinds of errors easy to identify.

Main menu IDs must be unique

Correct:

```
<mainmenu name="Mixing" id="mixing"/>  
<mainmenu name="MIDI" id="midi"/>
```

Incorrect:

```
<mainmenu name="Mixing" id="menu"/>  
<mainmenu name="MIDI" id="menu"/>
```

Commands must exist

If a command cannot be found, ReaCustomMenu ignores it and writes a warning to `ReaCustomMenu.log`.

REAPER must be restarted

After modifying the XML:

save the file → close REAPER → restart REAPER.

11. Troubleshooting

If a menu or command does not appear as expected, check:

`ReaCustomMenu.log`

The log file is located in REAPER's resource folder.

It contains XML loading errors and warnings about unknown commands.

`ReaCustomMenu` does not display error dialogs in order to avoid interrupting REAPER.

Summary

`ReaCustomMenu` essentially uses two files:

`reaper_ReaCustomMenu.dll`

→ the extension, installed in `UserPlugins`.

`ReaCustomMenu.xml`

→ the menu configuration, located in REAPER's resource folder.

The XML file supplied with the extension is intentionally minimal:

```
<?xml version="1.0" encoding="utf-8"?>
<menus></menus>
```

The user then adds their own menus and commands.

The XML file can be used to build:

- top-level menus;
- submenus;
- REAPER actions;
- scripts;
- SWS commands;
- separators.

Once the basic XML structure is understood, customizing the menus is simply a matter of adding, removing or moving the corresponding elements in the XML file.